

Các loại công cụ hỗ trợ trong phát triển phần mềm:

Giai đoạn / Mục đích	Công cụ / Loại	Ví dụ cụ thể	Ưu điểm	Hạn chế / cần cân nhắc
Thu thập yêu cầu / phân tích	- Công cụ vẽ sơ đồ / diagram / mockup / wireframe / prototyping - Quản lý tài liệu yêu cầu	Figma, Balsamiq, Miro, Draw.io Confluence, Notion, Google Docs	Giúp sinh viên hình dung thiết kế, tương tác UX/UI, dễ chỉnh sửa. Tài liệu trung tâm, versioned.	Học sinh mới — tốn thời gian làm quen. Cần khả năng hợp tác. Nếu dùng công cụ Online - phụ thuộc internet.
Quản lý dự án Scrum / backlog / task tracking	Jira, Trello, Taiga, Asana, Monday.com, ClickUp	- Jira rất mạnh, được dùng nhiều trong công ty, - Taiga có phiên bản mã nguồn mở, phù hợp dự án đào tạo. - Trello đơn giản, dễ hiểu cho nhóm nhỏ.	Làm quen quy trình Scrum: backlog, sprint planning, daily standups, burndown charts, học cách phân chia công việc, ưu tiên tính năng.	Chi phí bản trả phí nếu tính năng cao, nếu nhóm lớn, Jira có mức độ phức tạp. Nếu dùng nhiều tool khác nhau, overhead quản lý tool.
Version control / Mã nguồn & Code Review	Git + dịch vụ hosting repo + code review tool	Git + GitHub / GitLab / Bitbucket + Pull Requests / Merge Requests	Phổ biến trong công nghiệp, giúp học sinh học cách làm việc nhóm, xử lý merge conflict, code review.	Sinh viên chưa quen Git có thể gặp khó, cần hướng dẫn bài bản. Nếu codebase quá lớn hoặc học sinh không chăm, dễ sinh lỗi merge.
CI / CD / Build & Test tự động	Jenkins, GitLab CI/CD, GitHub Actions, CircleCI, Travis CI, Azure Pipelines	Ví dụ: GitHub Actions để build & test tự động mỗi lần push, GitLab CI nếu host repo trên GitLab.	Giúp đảm bảo chất lượng, phát hiện lỗi sớm, thực hành DevOps, sinh viên thấy được chuỗi từ mã nguồn tới deploy.	Thiết lập ban đầu có thể phức tạp, nếu máy chủ hoặc môi trường mạng yếu có thể chậm,

Giai đoạn / Mục đích	Công cụ / Loại	Ví dụ cụ thể	Ưu điểm	Hạn chế / cần cân nhắc
				nếu dùng miễn phí, hạn chế tài nguyên.
Kiểm thử (Testing) / QA	Unit test / Integration test / E2E test framework, công cụ giả lập / mock, test automation	JUnit, pytest, Selenium, Cypress, Postman (API test)	Học viên không chỉ code chạy được mà phải đảm bảo chất lượng, test giúp hiểu yêu cầu rõ hơn.	Thời gian học & thực hành dài, đôi khi khó config môi trường test, học sinh có thể bỏ qua nếu không có yêu cầu rõ ràng.
Triển khai / DevOps / Hạ tầng	Docker, Kubernetes, Virtual machines, Cloud providers (AWS / Azure / GCP), Heroku, Vercel, Netlify	Ví dụ dùng Docker để container hóa ứng dụng, Heroku / Vercel cho deploy web/app nhỏ, Kubernetes cho nhóm nâng cao	Giúp sinh viên hiểu môi trường thực, deploy, scale, hiểu các vấn đề thực tế như môi trường, cấu hình.	Chi phí cloud, nếu sinh viên chưa quen có thể gặp khó, nếu dùng nhiều công nghệ cùng lúc dễ gây overload.
Giao tiếp / Truyền thông nhóm / Quản lý kiến thức	Chat, công cụ họp online, Wiki / tài liệu nội bộ	Slack / Microsoft Teams / Discord, Zoom / Google Meet, Notion / Confluence / Wiki nội bộ	Tương tác tốt hơn, lịch sử trao đổi giữ lại, học sinh học cách phản hồi, phối hợp.	Nếu nhóm không có văn hóa sử dụng sẽ rất lãng phí, đôi khi công cụ quá nhiều gây rối, cần hướng dẫn rõ vai trò.
Theo dõi chất lượng mã & DevEX	Static code analysis, code linting, coverage, monitoring	SonarQube, ESLint / Pylint, Codecov, Sentry (giám sát lỗi runtime)	Giúp sinh viên chú ý code sạch, lỗi ít, maintainability, chuẩn bị cho thực tế công nghiệp.	Thiết đặt, cấu hình mất thời gian, cần có máy hoặc server hỗ trợ, kiến thức đủ sâu để hiểu các chỉ số.

Một số công cụ gợi ý cho các team làm software projects:

Dưới đây là những tổ hợp công cụ phù hợp với môi trường đại học, với sinh viên mới bắt đầu và cũng có thể mở rộng nếu nhóm nâng cao:

1. Tổ hợp cơ bản (nhóm nhỏ, học phần đầu):

- Quản lý dự án & Scrum: **Trello** hoặc **Taiga** phiên bản miễn phí — để làm quen backlog, sprint, story, tasks, board.
- Phiên bản yêu cầu / thiết kế: Google Docs / Notion cho tài liệu, Draw.io hoặc Figma để mockup UI.
- Version control: **Git + GitHub** (miễn phí cho sinh viên)
- Testing: unit test framework phù hợp ngôn ngữ giảng dạy (ví dụ: JUnit với Java, pytest nếu dùng Python)
- Giao tiếp nhóm: Slack hoặc Discord hoặc Google Meet/Zalo

2. Tổ hợp nâng cao hơn (nhóm lớn, yêu cầu cao, muốn thực tế hơn):

- Scrum & quản lý dự án: **Jira + Confluence**
- CI/CD: **GitHub Actions** hoặc **GitLab CI/CD** nếu host GitLab, xây dựng pipeline test tự động, deploy staging hoặc nhả tới Heroku / Vercel / AWS.
- Docker để container hóa ứng dụng dev/test.
- Code review + static code analysis: SonarQube hoặc dịch vụ tương tự.
- Monitoring/logs nếu deploy thực tế.

3. Tổ hợp “đầy đủ theo mô hình công nghiệp”:

- Scrum: Jira + Advanced plugins
- Version control + branching model (GitFlow hoặc trunk-based) + code review bắt buộc
- CI/CD đầy đủ: build → test tự động → deploy staging → kiểm thử tích hợp → deploy production
- Infrastructure: Docker / Kubernetes / IaC nếu có thể (Terraform hoặc CloudFormation)
- Chất lượng mã: SonarQube, coverage, security scan
- Ghi nhận lỗi runtime: Sentry hoặc tương tự

- Theo dõi hiệu năng nếu là ứng dụng web có người dùng thực

Các teams tự chọn lựa một tổ hợp phù hợp với nhóm mình